Intro To

Programming Architecture

Written By Adam Grouse

This paper is considered "currency" in Beta readings. Edits may be needed; readers' discretion is advised.

Abstract:

Programming is a complex science, as its development patterns depend on the way the user thinks. No one person thinks the same way as another programmer, creating a wide array of programs. A common solution is to follow architectural patterns; these solve the issue of Code that may be hard to read for an outside user by creating or standardizing a way of use without the need for over-complicated syntax in other programming languages. This brings a wave of Programming that follows architecture and allows for better follow-through on the programming. This also brings the ability to swiftly open closed programming, especially when users may or may not share the same way of thinking.

Introduction:

In programming, one of the most challenging and common issues is reviewing someone else's code; one of the best solutions is following a programming architecture. The general Purpose of this Article is to discuss programming architecture, specifically one commonly seen in software and database operations.

It is common knowledge that every programming language has its own syntax, or style of writing, that makes it different from every other programming language. While following standard syntax structure is one of the quickest and easiest ways to start off, it becomes less and less effective the more complex a program gets.

Studies have shown that programs that follow an architectural pattern are easier to understand and interpret for humans. This allows ease of understanding amongst teams, so instead of working off of 30 different classes, objects. Architectural principles and patterns allow anyone who knows the language and the pattern or principle to accurately interpret a program written.

This article is meant to be human by nature and not an exact research article style, as this article discusses what programming architecture is and use cases going over the three major programming architecture styles:SOLID,CRUD, and KISS. What makes them work, what each one does and how to use them.

Background/Context:

In programming, code can become unreadable because it reflects an individual's thought process and algorithmic and game theory approaches, which may not be clear to others. If someone does not understand your reasoning, it is hard for them to follow your code. Programming often relies on abstraction, so as long as a library serves its intended purpose, details of how it works are often overlooked. However, the culture of "it ran on my machine" leads to problems when code or libraries do not work on other systems, forcing users to resolve compatibility issues themselves. Code that does not follow common patterns or uses unusual syntax or libraries makes maintenance and understanding harder. If a library requires reworking but does not follow standard practices, it is unreasonable to expect others to fix it easily.

What Is Programming Architecture:

Programming Architecture are rules and expectations that are seen in high-level readability programs. Unlike syntax Architecture is not technically necessary as it is high level requirements for human readability. A good example is the statement "Seth sets sets sets so Rhett's sets set." While it is technically a grammatically accurate sentence, it is hard to read and understand; this is a good example of how programming can become hard. Programming architecture regulates how things should be managed and handled using the original sample from before, a structural pattern turns that statement into "Seth sets up sets of sets so Rhett's sets can set". It's more human-readable without changing the overall meaning of the outcome; it becomes more structured and understandable from onlookers.

Overview of Common Architectural Styles

Programming Architecture comes in many different forms and regulations; the most common three are as follows: SOLID, CRUD, and KISS. SOLID is an acronym that outlines five principles: Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion. These principles guide the design of software architecture to make systems easier to maintain and extend; it also holds the most data and open-endedness. CRUD stands for Create, Read, Update, and Delete, which relate to fundamental database operations. Lastly, KISS means Keep It Simple, Stupid, which suggests simplifying complex programs into more manageable APIs.

Architectural Pattern 1: SOLID

The SOLID pattern of Architecture is the staple of software architecture patterns due to how much it covers; ITs big on openness and expandability (it's also why the SOLID section is so long)

The S in SOLID stands for the Single Responsibility Principle; every class should have a single purpose. This means everything is needed for one reason and one reason only, and it must do that Job well. An example of this is a Fork, a spoon, and a Spork. A fork is great for things that need to be poked, like salads, but it doesn't work when trying to have soup. A spoon is good for holding liquid like soup, but it isn't that good for salads. A Spork can poke salad and hold soup, but it doesn't necessarily do either. Well, limitations are added for each to make the whole make sense. The single responsibility tries to avoid the spork issue by saying you need a fork and a spoon, and a spork is not allowed.

O stands for open closed principal. The open-closed principle states everything should be open to extension but closed for modification. A class or object should be able to be expanded on, but it can't be changed. Like appending to a value, the initial value isn't changed, but is expanded to hold or do more.

An example is that brain surgery isn't needed to put on a hat. You shouldn't need to change the initial value, but build around it like a fitted hat.

L stands for Liskov Substitution Principle. A driverided, child, or class that inherits from a bigger class can be substituted for the parent class. This means that a class built off of another class can substitute for that class usage. A programming example of this is int and Integer in Java, where the int class can be called and treated like an Integer and is compatible with all int things. A Realife example of this is when you learn to drive you should be able to drive any car afterwards. This idea stands that if you know how to use a car, any other car that drives from that initial car can be used.

I stands for Interface Segregation. The Interface Segregation principle states that an Object or entity should not be forced to use an interface relevant to them. For context Interfaces define what should be in this class and other classes like it. An object called sandwich should take on example interfaces like UserObject, Edible,FoodType, but if FoodType contains the NeedsToBeCooked method, it's better to make another called sandwich type, which doesn't contain cooking instructions for a sandwich. A human Example is if your phone has a usb-c it should not need Ethernet, HDMI, DC port charging support just because USB-C is a port. It enables a more strategic and open use of interfaces, making sure everything in an interface is applied to support polymorphism.

D stands for Dependency Inversion. The Dependency Inversion principle states that high-level modules should not depend on low-level modules. Both should depend on abstractions. This means a high-level class should not need a lower module, like an integer, to run properly. A Human example of this is lets say Alex is the lead programmer on a project, he's the only one doing a part, if he quits. There's no one to take its place; it all crumbles down. This does not follow the dependency inversion principle because everything relies on one low-level thing. But now, if the Job is Abstracted and it's not Alex’s job but the Job of a high-level programmer, anyone can take over as needed.

Overall, SOLID is the most common pick because it's universal. It doesn't say this is how we should and this is what works best with it; it's rules and expectations not demands, easy to implement

when writing programs and open-ended enough for creative liberties.

Architectural Pattern 2: CRUD

Architectural Pattern 2 is CRUD, which goes alongside SOLID as a way to Work with Databases in a patternized way.

C stands for Create. Create means an object that works with databases must be able to create/seed a Database With propel and reliable Data. Making sure there No many to Many relationship or making sure Locations have standardized names, making the relationship reliable to update, and Readability

R stands for Read. Read means Objects meant to read from a Database can also properly understand and parse the needed information. Making sure the readed Data is Reply Read from the Database.

U stands for Update. Update means the database can be properly updated. An object built for a database should be able safely update the Database with limited resources and be able to handle and parse the input data for Update.

D stands for Delete. Delete means to remove an object from the table, safely and securely, making Sure Foreign/Primary keys are properly handled or disposed of.

These CRUD Operations, while not as much as Solid Principles, outlines the Structure needed for objects to successfully work with Databases; as especially working with NoSQL non-SQL languages, many commands can be convoluted or unused. Following CRUD tries to ensure everything is needed.

Architectural Pattern 3: KISS

KISS stands for Keep It Stupid Simple. KISS fills in the options that SOLID or CRUD may not cover. The idea of KISS is to keep it simple, keep it manageable. This emphasizes designing systems with the least amount of complexity. It encourages people to avoid unnecessary abstractions, overengineering,

and features that are not immediately required. Many developers ignore KISS due to it coming off as sloppy; but it's great for prototype programming where effect and readability may not be a must or in an environment where refactoring is already planned and a prototype is needed. Kiss beats other architectures via speed, but in realistic situations where a program needs to be run quickly, effectively and readable, KISS starts to fall apart.

Conclusion:

Programming Architecture exists to make code understandable, maintainable and flexible by following architectural patterns such as SOLID, CRUD, and KISS. programmers can reliably work with each other by following the pattern of code- even if everyone has a different way of reading. While this article is meant to be partially playful, the principles are real and used within the real world and applying them can make projects smoother and less confusing for future readers.